

SID Analysis Codebase — Session Notes

Conversation with Claude · May 28, 2026

Q: Explain the Python programs to me

Overview

These three Python scripts form a pipeline for analyzing **Sudden Ionospheric Disturbances (SIDs)** — disruptions to Earth's ionosphere caused by solar flares, detected by amateur radio observers monitoring VLF (Very Low Frequency) signals.

Script 1: `1goesevents.py` — GOES Solar Event Extractor

This script processes raw solar event data from NOAA's GOES satellite and extracts relevant events.

What it does:

- Asks the user for a month/year (e.g. "Jun 11") and a base directory path
- Scans a folder of daily event files named like `yyyymmddEvents.txt`
- Filters lines matching a requested event qualifier — either **FLA** (solar flares) or **XRA** (X-ray events)
- Extracts specific fields (date, time, event type) by fixed character positions in each line
- Writes one sorted output file per qualifier (e.g. `JUN_11FLA.txt`, `JUN_11XRA.txt`)
- Output files group events by day, with a blank line between days

Script 2: `2goesSumm.py` — XRA Event Summarizer

This script takes the XRA output from Script 1 and produces a statistical summary.

What it does:

- Reads the `mmm_yyXRA.txt` file produced by Script 1
- Counts flares **by class** (A, B, C, M, X — the standard solar flare classification scale) for each day of the month
- Writes a CSV where each row is a flare class and each column is a day, giving a quick month-at-a-glance matrix
- This CSV is intended as input for further analysis in R

Script 3: `3reporter_main.py` — SID Correlation Analyzer

This is the main and most complex script. It correlates amateur observer SID reports against each other and against the GOES solar event data.

What it does, step by step:

1. **Setup** — asks for month/year, whether to use GOES data, a minimum observer quality threshold, and whether to update the observers database
2. **Reads observer .dat files** — via a file picker dialog, creating OBSERVER objects containing their name, location, quality rating, and a list of events with timing and importance

3. **Correlates observer-to-observer** (CorrelateObservers) — finds events reported within 5 minutes of each other across multiple observers, averaging their peak times and importance ratings
4. **Secondary correlation** (CompareObserversToCorrList) — catches events within 15 minutes of an already-correlated event
5. **GOES cross-correlation** — optionally matches observer events against the FLA/XRA data from Scripts 1 & 2 (within 15 minutes)
6. **High-quality observer inclusion** — optionally includes uncorrelated events from trusted observers above a quality threshold
7. **Generates output reports** — several file formats including an NGDC-format report, a database file, and observer quality summaries

Supporting Files

- **SIDAnalObservers.ini** — a registry of ~80 amateur observers worldwide, storing names, locations, and quality ratings (0–10 scale, updated over time based on how often their reports correlate with others)
- **SIDAnalControl.ini** — basic configuration (base directory, array size limits)
- **SIDAnalStations.ini** — maps VLF transmitter call signs (NAA, NLK, etc.) to their frequencies

Notable Code Issues

Since this is legacy Python 2 code, a few things worth knowing:

- All three scripts use **raw_input()** — Python 2 only; needs updating for Python 3
- Script 3 uses **ConfigParser** (Python 2 name; it's *configparser* in Python 3) and **Tkinter** (similarly capitalized differently in Python 3)
- Typo bug in *DetectHiQualNonCorrelatedEvents*: `Control['HiQualLimit==0']` should be `Control['HiQualLimit'] == 0`
- The `types.sort()` call in Script 2 won't work in Python 3 (dict keys aren't directly sortable that way)

Q: What sub-directories does it use?

Based on the code, the scripts expect this directory structure:

```
<baseDir>/
■■■ MMM_YY/                                e.g. JUN_11/
    ■■■ eventlistings/                      (Script 1 INPUT - daily GOES event files)
    ■ ■■■ yyyymmddEvents.txt
    ■ ■■■ yyyymmddEvents.txt
    ■ ■■■ ...
    ■■■ Data Received/                      (Scripts 1,2,3 OUTPUT; Scripts 2,3 INPUT)
        ■■■ MMM_YYFLA.txt                   (Script 1 output / Script 3 input)
        ■■■ MMM_YYXRA.txt                   (Script 1 output / Scripts 2,3 input)
        ■■■ MMM_YYXRASumm2.csv              (Script 2 output)
        ■■■ [various report files]          (Script 3 output)
```

Notes:

- **baseDir** is whatever path the user enters at the prompt, defaulting to the current directory
- **MMM_YY** is formatted like *JUN_11* — three uppercase month letters, underscore, two-digit year
- **Data Received** has a space in it, which could cause issues on some systems depending on how it's called

- Script 3 also expects the observer *.dat* files somewhere the user can navigate to via the Tkinter file picker — their location isn't constrained to this structure
- The *SIDAnaObservers.ini* and *SIDAnaStations.ini* files are expected in the **current working directory** by default, though Script 3 accepts command-line arguments to override their paths